WALL-E Live Demo Guide & Presentation Script

Audience: VinnovateIT Technical Interviewers | **Demo Duration:** 3 Minutes Max | **Platform:** Local Terminal & Browser

Pre-flight checklist, spoken script table, 'wow moment' walkthrough, troubleshooting, and offline backups.

---

1. Before-the-Demo Pre-Flight Checklist

Follow these exact steps before you enter the interview room so your demo runs flawlessly in 5 seconds:

1. **Open Terminal in Project Root:** Open PowerShell or VS Code terminal in c:\Programming\Projects of antigravity\project_5_aiml_vinnovateit_walleai.
2. **Activate Virtual Environment:** Run .\.venv\Scripts\Activate.ps1. Verify (.venv) appears in prompt.
3. **Zero API Keys Needed:** The entire simulation and math run locally in pure Python. No internet connection, OpenAI keys, or external servers required!
4. **Pre-Open Generated Visuals:** In your file explorer, have outputs/mission_medium_seed7/summary.png and run.gif ready in an image viewer tab.
5. **Test Live Command:** Run python -m walle run --config configs/mission_easy.yaml --ascii once to ensure it executes smoothly (takes 2 seconds).
6. **Clean Screen:** Close WhatsApp, Discord, and personal tabs. Put laptop in 'Do Not Disturb' mode.

2. Three-Minute Live Demo Script (Two-Column Table)

Start with your one-line intro, perform the screen action, and read the spoken dialogue naturally:

What You Do On Screen	What You Say Aloud to Interviewers
Open terminal showing clean project directory. Have command typed: python -m walle run --config configs/mission_easy.yaml --ascii	"To demonstrate how WALL-E works in real time, I'm running an autonomous search mission with live ASCII terminal telemetry. Notice how the agent starts at charger C with 100 battery."
Press Enter. The live ASCII frames stream on screen. W moves, heat map characters change, step counters tick.	"As WALL-E moves east and south, watch the belief heat map on the bottom right. The dots and symbols show probability density. In Step 12, he executes a SIGNAL SCAN, and the posterior sharply concentrates around EVE's coordinates."
Point to terminal output as strategy switches: EXPLORE -> TRACK -> VERIFY -> DECLARE.	"Here you see the finite state machine in action: he transitioned from EXPLORE into TRACK, verified the coordinates, and declared EVE active at position (7, 11) with only 1 cell error, consuming just 15 battery units."
Open outputs/mission_medium_seed7/summary.png on screen. Show the 6 panels.	"Now let's look at a more complex mission on a 20x20 grid. This 6-panel dashboard shows WALL-E's complete journey. Panel 1 is the 2D path map around obstacles. Panel 2 shows how belief entropy drops from 10 bits down to zero."
Point to Panel 5 (Energy & Return Reserve Curve) in summary.png.	"This bottom panel is our critical safety guarantee: the blue line is current battery, and the orange dashed line is the minimum return reserve to reach the charger. Notice the blue line never touches the orange line—stranding is mathematically prevented."
Switch back to terminal and run: python -m walle verify outputs/mission_medium_seed7	"Finally, to prove that our simulation is 100% honest and never cheated, I built an independent verifier. Running walle verify replays every step, checking energy conservation and wall collisions. It passes with zero violations."

3. The 'Wow Moment' of Your Demo

The Ultimate Wow Moment:
Point to **Panel 5 (Battery vs Return Reserve)** in summary.png and say:
*'Many student robotics simulations let agents run until they succeed or run out of juice. In our project, WALL-E calculates the Breadth-First Shortest Path to the nearest charger before every single step. In over 200 randomized stress tests with tight batteries, WALL-E had exactly **ZERO stranded failures (0.0%)**. Even if sensors lie or decoys distract him, his survival invariant is mathematically protected.'*

Why this impresses interviewers: AI/ML evaluators see dozens of models with high accuracy that fail in deployment. Showing a formal, guaranteed safety constraint proves you understand *reliability in production*, not just toy accuracy.

4. Exact Sample Commands & Expected Outputs

Copy-paste these exact commands in your rehearsal:

Purpose	Exact Command to Type	Expected Output / What to Look For
1. Fast Live ASCII Run	python -m walle run --config configs/mission_easy.yaml --ascii	Prints 15 grid frames live in 2.5s. Ends with: Outcome: DECLARED, Success: SUCCESS, Location Error: 1.00 cells.
2. Independent Verification	python -m walle verify outputs/mission_medium_seed7	Replays 21 steps. Prints: WALL-E TRACE REPLAY VERIFICATION: mission_medium_seed7 -> Status: [PASSED].
3. Render Visuals	python -m walle viz --run outputs/mission_medium_seed7	Generates summary.png, run.gif, path_map.png, path_3d.png, belief_evolution.png, metrics_dashboard.png.
4. Quick Benchmark Check	pytest tests/test_benchmark.py -v	Runs in 20 seconds. All 8 tests pass, proving calibration bin means, CSV reconciliation, and paired seeds.

5. Features to Skip or Avoid During the Live Demo

Do not attempt these during your 3-minute window:

- **DO NOT run the full 100-mission benchmark live:** Running python -m walle benchmark --n 100 evaluates 300 missions and takes about 2 to 3 minutes. It will burn your entire interview time! Instead, show the pre-generated outputs/benchmark/benchmark_chart.png.
- **DO NOT run the 200-seed calibration script live:** scripts/calibration_study.py takes ~4 minutes. Point to outputs/calibration.md and outputs/calibration.png instead.
- **DO NOT run the full pytest suite with fuzz tests:** Running pytest -q executes 200 randomized fuzz tests taking 13 minutes. If asked about tests, run fast tests: pytest tests/test_benchmark.py (20 seconds).
- **DO NOT edit YAML configs live:** A typo in indentation could cause a YAML parse error while under pressure. Use the existing validated configs.

6. If Something Breaks: Quick Fixes & Calm Spoken Lines

Stay calm. Interviewers love seeing a student troubleshoot professionally:

Failure Scenario	Immediate Quick Fix	What to Say Calmly to Interviewers
'python : command not found' or module error	Run: .\.venv\Scripts\Activate.ps1 or use .\.venv\Scripts\python.exe directly.	"Let me quickly activate the local virtual environment where NumPy and dependencies are installed."
Terminal execution is lagging or frozen	Press Ctrl + C to terminate, then rerun without --ascii: python -m walle run --config configs/mission_easy.yaml	"The terminal ASCII renderer buffer had a small delay; let me run the headless mission directly—it takes under 1 second."

Image file won't open in viewer	Open it directly in your browser: drag outputs/mission_medium_seed7/summary.png into a Chrome or Edge window.	"Let me drag the generated PNG directly into Chrome for a crisp high-resolution view."
Accidental typo in command	Press Up Arrow on your keyboard to recall the last working command and press Enter.	"Pardon the typo; pulling up the validated command from terminal history."

7. Backup Plan: What If Your Laptop Completely Glitches?

If your laptop restarts or crashes, don't panic. You can present directly from pre-rendered static artifacts on your phone or pen drive:

1. **Pre-Rendered Animated GIF:** Keep outputs/mission_medium_seed7/run.gif saved on your phone or Google Drive. You can show WALL-E moving in the browser tab.
2. **Summary Dashboard:** outputs/mission_medium_seed7/summary.png contains all 6 panels (Path Map, Entropy, Belief Marginal, Confidence, Battery vs Reserve, Evidence Log).
3. **Benchmark Chart:** outputs/benchmark/benchmark_chart.png compares all 3 architectures on Success, Error, Battery, and Stranded rate.
4. **Markdown Audit Report:** outputs/mission_medium_seed7/report.md contains the complete audit trail and evidence ledger.
5. **Verbal Walkthrough:** If screen is completely dead, open the code in VS Code and walk through the 5 functions from Section 7 of INTERVIEW_PREP.pdf.

8. Questions Interviewers Ask DURING the Demo

Interviewer: 'What do the symbols like @, #, %, and : mean in the terminal?'

Answer: ***"The '#' are obstacles/walls. 'C' is a charging station. 'W' is WALL-E. The dots, colons, pluses, and '@' symbols represent the belief intensity ramp from low to high probability. '@' is the peak hypothesis where the agent thinks EVE is."***

Interviewer: 'Where does the initial signal come from?'

Answer: ***"At step 0, EVE emits a faint broadcast before the mission starts. In generated mode, the environment samples this signal from EVE's actual location using the physical sensor equation with noise."***

Interviewer: 'How does WALL-E know when to stop and declare?'

Answer: ***"In agent.py, the agent monitors his location confidence and status confidence. When location confidence exceeds 0.90 and status confidence exceeds 0.70—and there are no pending decoy confirmations—he triggers the DECLARE action."***

Interviewer: 'What happens if battery runs completely out?'

Answer: ***"Because of our centralized return reserve guard, battery never runs out in normal missions. If time runs out (250 ticks), the runner forces a declaration using his current best MAP estimate."***

Interviewer: 'Why does the battery line go up in the middle of mission_hard_decoys?'

Answer: ***"Because on that hard mission, WALL-E detected his battery dropping toward the reserve, navigated back to charger C, executed 5 RECHARGE actions (+20 battery each), and then continued the mission fully charged."***

9. One-Page 'Demo Day' Final Checklist

DEMO DAY MORNING CHECKLIST (Print or Review on Phone):

[ ] Laptop charged to 100% and power brick in bag.
[ ] Project cloned and tested in local directory.
[ ] VS Code open with terminal in (.venv).
[ ] Browser window open with tabs: summary.png and run.gif.
[ ] Terminal font size bumped up (Ctrl + Plus) so interviewers sitting 3 feet away can read it easily.
[ ] Notifications, Slack, WhatsApp, and auto-sleep disabled.
[ ] Backup copy of INTERVIEW_PREP.pdf, DEMO_GUIDE.pdf, and PNGs on your phone or pen drive.
[ ] Remember to take a deep breath, smile, speak at a steady pace, and enjoy sharing what you learned!